

MILITARY ASSET MANAGEMENT SYSTEM

MAMS - Project Documentation

Project	Military Asset Management System (MAMS)
Frontend	React with Vite
Backend	Spring Boot with Java 17
Database	PostgreSQL
Authentication	JWT-based authentication
Development setup	Localhost

Prepared for project submission

This document describes the current working version of the project and the local setup used during development and testing.

1. Project Overview

MAMS (Military Asset Management System) is a web application for keeping track of military asset-related information such as bases, purchases, transfers, assignments and expenditures. The current version has a React frontend connected to a Spring Boot REST backend and a PostgreSQL database.

The project has two main parts. The frontend is a React application where the user logs in and interacts with the system. The backend is a Spring Boot application that provides REST APIs and handles authentication, business operations and database access.

Assumptions

- The application is being used in a controlled environment and the database is reachable by the backend.
- Users who access protected features have valid login credentials.
- The current demonstration data is small; the tested database contains a Bangalore Base.

Current limitations

- The project is currently a local development setup rather than a production deployment.
- The amount of sample data is limited, so the dashboard can show zero values until transactions are added.
- Only the screens and backend operations implemented so far are available; additional management features can be added later.

2. Tech Stack & Architecture

The project uses a straightforward full-stack structure. React handles the browser interface, Spring Boot provides the REST API and PostgreSQL stores persistent data.

Frontend - React with Vite

The frontend was created using Vite and React. React is used for the pages and reusable UI components, while React Router is used to move between pages such as Login, Dashboard and Bases. Vite provides the development server and production build process.

API communication - Axios

Axios is used by the frontend to call the Spring Boot API. A common Axios instance points to `http://localhost:8080/api`. A request interceptor reads the JWT from `localStorage` and adds it as a Bearer token to protected requests.

Backend - Spring Boot and Java 17

The backend is a Spring Boot application written for Java 17. It exposes REST endpoints for authentication, dashboard information, bases and the asset-related operations. The code is separated into controllers, services, repositories, entities, security and DTOs.

Database - PostgreSQL

PostgreSQL is the relational database used by the project. It is a good fit here because the system has related entities and transactions that need consistent relationships, such as a purchase belonging to a base and an equipment type. In the current local setup, the backend connects to PostgreSQL on port 5432 and uses the mams_db database.

Overall request flow

```
Browser (React)
↓
Axios request + JWT
↓
Spring Boot REST API
↓
Security / Controller / Service
↓
Repository / JPA
↓
PostgreSQL
```

3. Data Models / Schema

The application is organized around users, bases and asset movement records. The backend uses JPA entity classes to represent the database tables.

Main entities

User - Stores application user information used during authentication.

Role - Represents the role associated with a user.

Base - Stores a base name and location. The current test data contains Bangalore Base.

EquipmentType - Stores equipment types or categories.

Equipment - Represents equipment or asset information managed by the system.

Purchase - Records equipment purchased for a base, including quantity, purchase date and supplier.

Transfer - Records movement of equipment between bases.

Assignment - Records equipment assigned to personnel.

Expenditure - Records equipment or quantities that have been expended.

AuditLog - Stores activity information used for tracing actions in the application.

Example relationship

The Purchase entity is connected to both Base and EquipmentType. A purchase contains the base, equipment type, quantity, purchase date and supplier information. This relational structure allows the dashboard and transaction pages to work with the same underlying data.

4. RBAC Explanation

Role-Based Access Control (RBAC) is used so access can be controlled according to the user's role. The current administrator login is the main account used while testing the platform. The JWT generated by the backend contains the username and role claims.

For the tested administrator account, the JWT contained the username claim for admin and the role claim ADMIN. Spring Security can use this information when deciding whether a protected request should be allowed.

How access is enforced

- The user sends username and password to POST /api/auth/login.
- The backend returns a JWT when the credentials are correct.
- The frontend stores the token and sends it as a Bearer token for protected requests.
- The backend security layer validates the token before allowing access to protected endpoints.
- Requests without valid authorization can receive HTTP 401 or 403 responses.

5. API Logging

The project includes an AuditLog entity and repository for keeping track of application activity. This gives the system a separate place for recorded activity instead of relying only on the current state of the asset tables.

The backend exposes audit-log endpoints, and these endpoints are protected in the same way as the other APIs. During testing, the audit-log request was made using the JWT obtained after login.

Audit endpoints

- GET /api/audit-logs - retrieves audit-log records.
- GET /api/audit-logs/user/{username} - retrieves logs for a particular username.
- GET /api/audit-logs/entity/{entityType} - retrieves logs related to an entity type.

6. Setup Instructions

Backend

Open PowerShell in the backend project folder and start the Spring Boot application using the Maven wrapper included with the project.

```
cd C:\Users\Akarsh\Desktop\MAMS\backend
.\mvnw.cmd clean package
.\mvnw.cmd spring-boot:run
```

The backend used during testing runs on port 8080. The API base path is http://localhost:8080/api.

PostgreSQL database

Start the PostgreSQL service and make sure the database used by the project is available. The current local setup uses PostgreSQL on localhost:5432 with the database name mams_db. The Spring Boot application connects to this database through its datasource configuration.

```
Host: localhost
Port: 5432
Database: mams_db
```

Frontend

Open another PowerShell window in the frontend folder.

```
cd C:\Users\Akarsh\Desktop\MAMS\frontend
npm install
npm run dev
```

Vite starts the frontend at <http://localhost:5173/>. The Axios client points to <http://localhost:8080/api>.

Running everything

- Start PostgreSQL.
- Start the Spring Boot backend and wait for port 8080.
- Start the React/Vite frontend on port 5173.
- Open the frontend in a browser.
- Log in using the credentials in Section 8.
- The frontend then uses the JWT for protected requests.

7. API Endpoints

The following are the main endpoints present in the current backend controllers. Protected endpoints are accessed with the JWT returned from the login API.

Authentication

```
POST /api/auth/login
Body: { "username": "admin", "password": "Admin@123" }
```

Dashboard

```
GET /api/dashboard
Authorization: Bearer <JWT>
```

The dashboard endpoint returns the inventory summary used by the React dashboard, including opening balance, purchases, transfer in, transfer out, assigned, expended, net movement and closing balance.

Bases

```
GET /api/bases
Authorization: Bearer <JWT>
```

The tested response included the Bangalore Base record.

Other protected endpoints

```
GET /api/assignments
GET /api/audit-logs
GET /api/expenditures
GET /api/purchases
GET /api/transfers
```

These requests were tested with the JWT returned from the login endpoint.

8. Login Credentials

The following account was used for local testing of the current application:

Administrator account

```
Username: admin  
Password: Admin@123  
Role: ADMIN
```

These credentials are included for project evaluation and local demonstration. They should be changed before any real deployment.

Testing completed during development

The login API successfully returned a JWT token. Using that token, the dashboard API returned its summary and the bases API returned the configured Bangalore Base. The assignment, audit-log, expenditure, purchase and transfer GET endpoints were also invoked during API testing. The React frontend was successfully started using Vite.

The current implementation is ready to be demonstrated locally, while the remaining frontend screens can be added as the project continues.